AI-SOC Analyst Agent

Project Title: AI-Powered SOC Analyst Automation Lab

1. Project Overview & Architecture

This project simulates an automated Tier-1 Security Operations Center (SOC) pipeline. It utilizes Python to capture live network traffic, dynamic thresholding to identify malicious activity across multiple protocols (ICMP, SSH), and an LLM-powered AI agent to ingest the telemetry and generate structured triage reports.

Infrastructure:

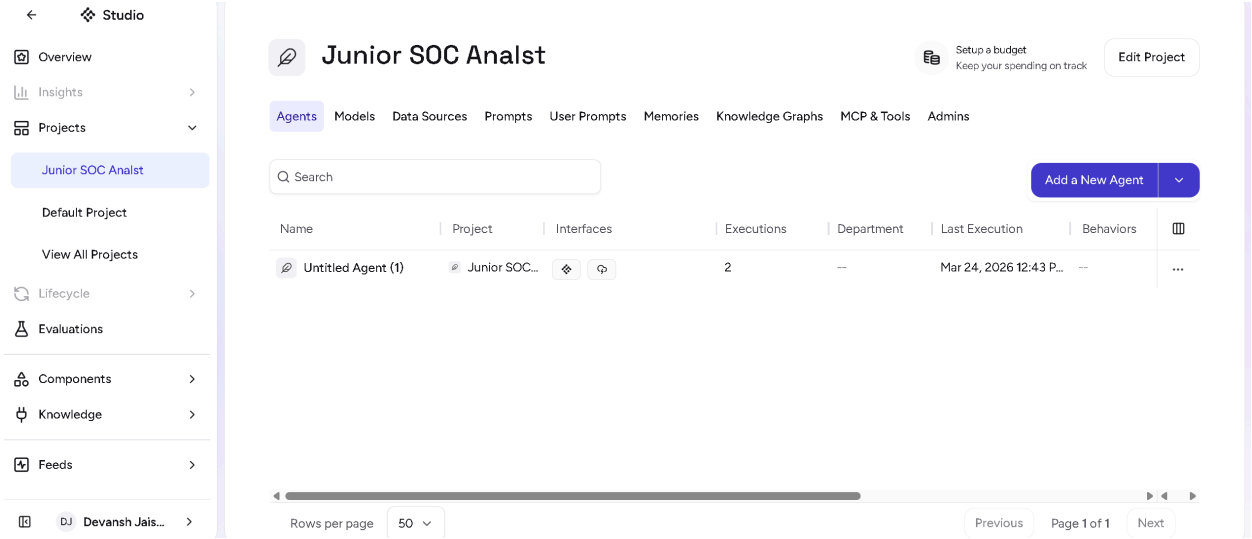
- **Target Server (Defender):** Kali Linux (`192.168.56.20`) running the Python automation script.
- **Threat Actor (Attacker):** Ubuntu Server (`192.168.56.102`) generating malicious traffic.
- **AI Engine:** Airia.ai running a custom-prompted GPT-5 Nano model.

---

2. Phase 1: Building the AI Agent

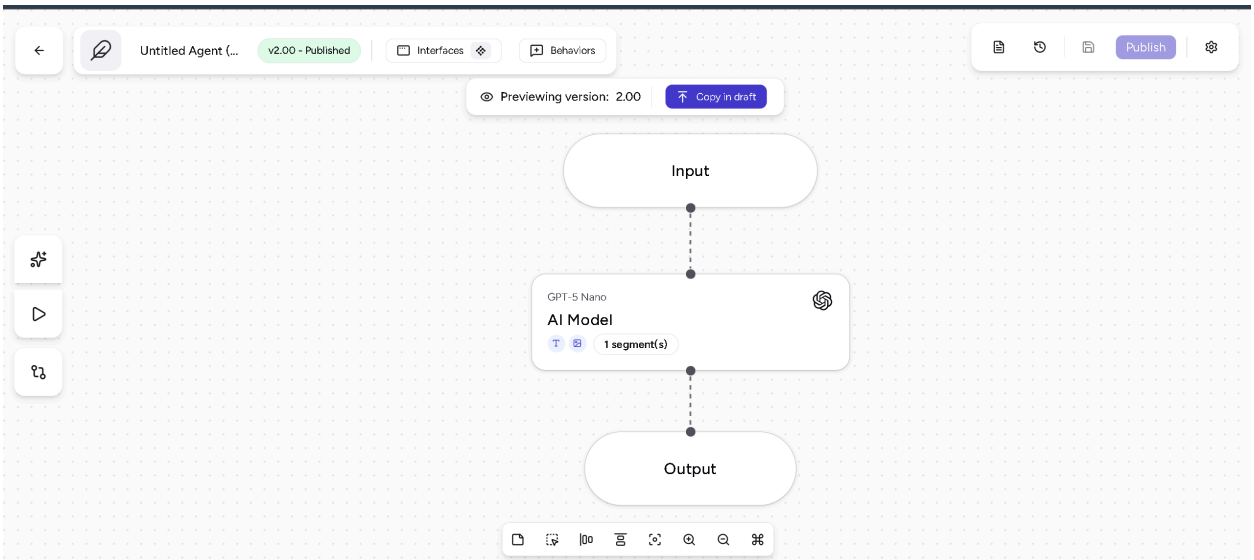
The first step was establishing the AI agent and configuring its internal SOC playbook to ensure strict JSON output and accurate risk scoring.

> *The Airia Studio dashboard showing the "Junior SOC Analst" project)*



The agent's logic flow was designed to take raw JSON telemetry as input, process it through the predefined SOC triage instructions using the GPT-5 Nano model, and return a structured JSON response.

> *The Airia workflow canvas showing Input → AI Model → Output)*

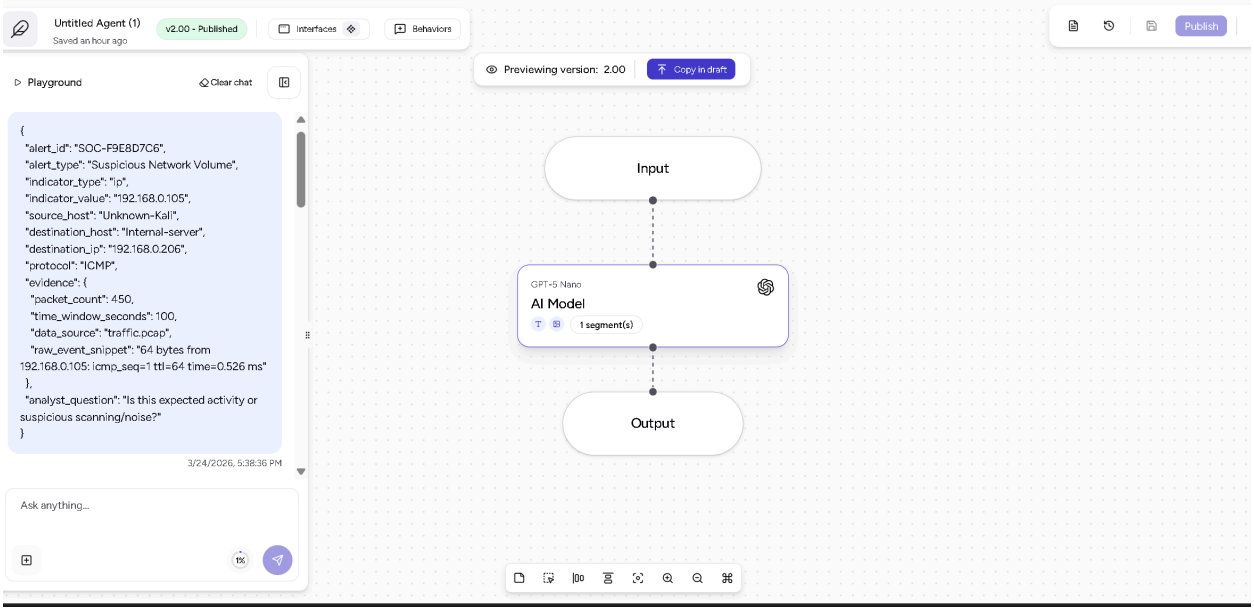


3. Phase 2: Testing the Playbook Logic

Before deploying the agent to the live network, the triage logic was tested manually in the Airia Playground. A mock JSON alert simulating an ICMP flood was

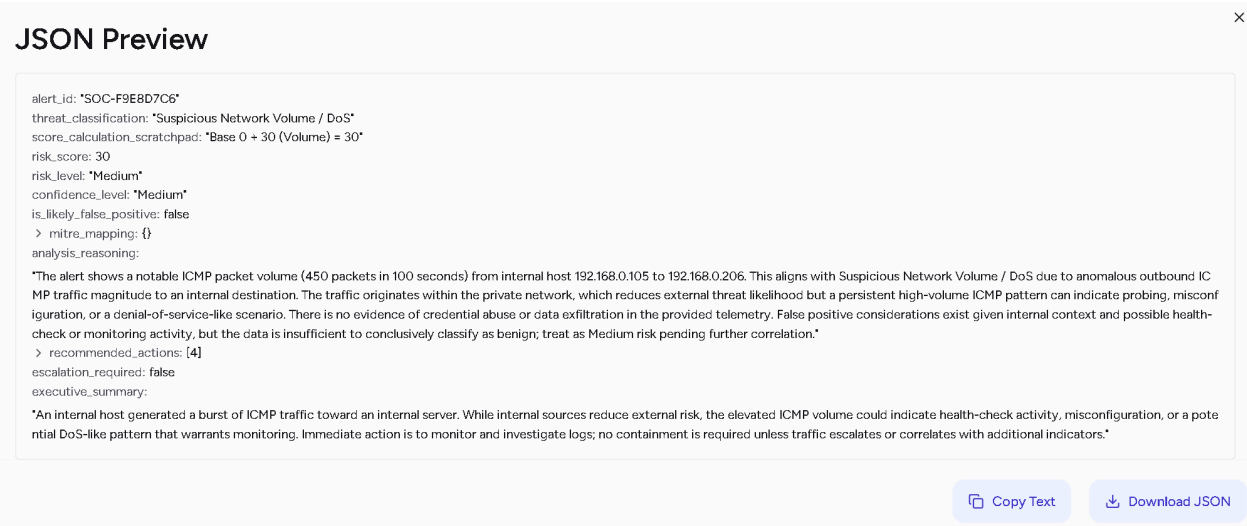
fed into the system.

> *The Airia Playground showing the mock JSON input pasted in the chat)*



The AI successfully parsed the data, calculated the risk score accurately using the designated "scratchpad" technique, mapped the activity to MITRE ATT&CK, and output a flawlessly formatted JSON triage report.

> *The JSON Preview screen showing the Medium risk score and classification)*



4. Phase 3: Live Attack Simulation

With the AI validated, the live environment was staged. Malicious traffic was generated from the Ubuntu Threat Actor machine aimed at the Kali SOC Target.

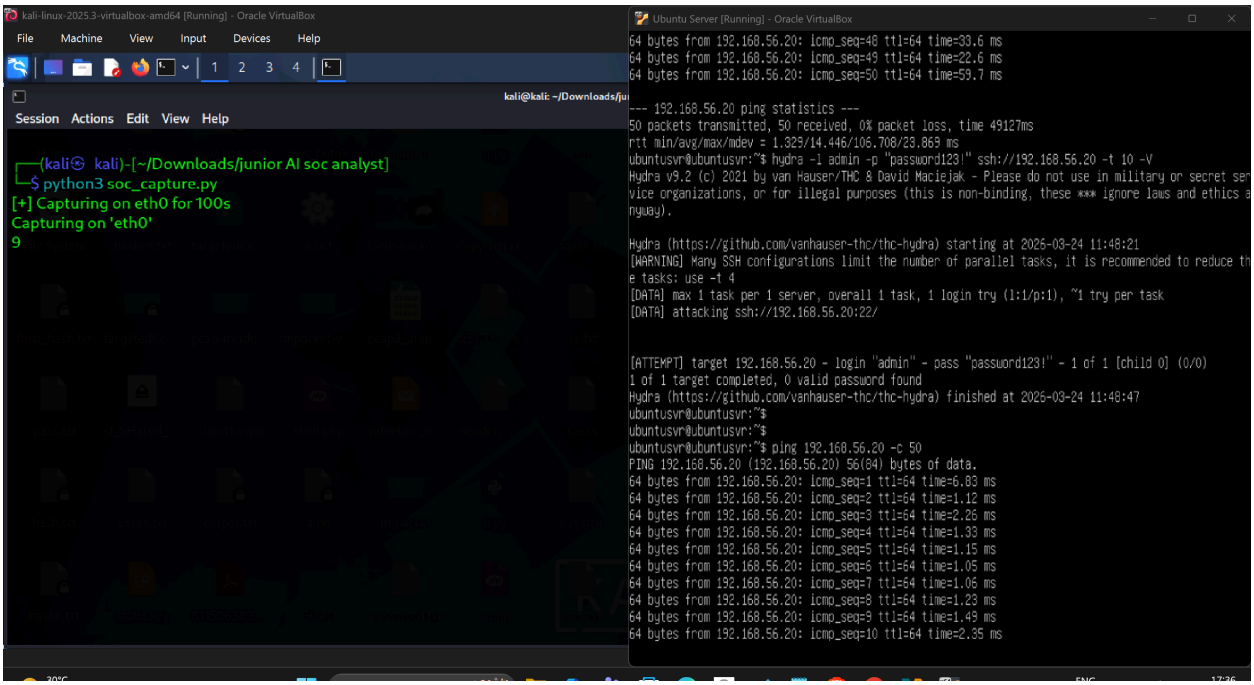
To test the multi-protocol detection capabilities, an SSH brute-force attack was launched using Hydra, followed by a standard ICMP ping flood.

> *The Ubuntu terminal showing the `hydra -l admin...` command)*

```
ubuntusvr@ubuntusvr:~$ hydra -l admin -p "password123!" ssh://192.168.56.20 -t 10 -V
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret ser
vice organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics a
nyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-24 12:07:00
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce th
e tasks: use -t 4
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.56.20:22/
[ATTEMPT] target 192.168.56.20 - login "admin" - pass "password123!" - 1 of 1 [child 0] (0/0)
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-24 12:07:04
ubuntusvr@ubuntusvr:~$ _
```

> *The split-screen showing Kali listening on the left, and Ubuntu attacking on the right)*



---

5. Phase 4: Automated Pipeline (Capture & Conversion)

On the defending Kali machine, the `soc_capture.py` script acts as the automated data pipeline. First, it captures the raw network packets using `tshark`.

> *The green Kali terminal output showing the raw `traffic.pcap` ICMP Echo requests)*

```
┌──(kali㉿kali)-[~/Downloads/junior AI soc analyst]
└─$ tshark -r traffic.pcap
 1 0.000000000 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=3/768, ttl=64
 2 1.003121053 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=4/1024, ttl=64
 3 2.005545195 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=5/1280, ttl=64
 4 3.007927590 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=6/1536, ttl=64
 5 3.148392139 PCSSystemtec_3f:bd:d2 → PCSSystemtec_1f:b7:23 ARP 60 Who has 192.168.56.20? Tell 192.168.56.102
 6 4.010970980 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=7/1792, ttl=64
 7 5.012960484 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=8/2048, ttl=64
 8 6.015784900 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=9/2304, ttl=64
 9 7.019670840 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=10/2560, ttl=64
10 8.021023084 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=11/2816, ttl=64
11 9.025996697 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=12/3072, ttl=64
12 10.027068825 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=13/3328, ttl=64
13 11.027876744 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=14/3584, ttl=64
14 12.032002245 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=15/3840, ttl=64
15 13.035090031 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=16/4096, ttl=64
16 14.039614984 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=17/4352, ttl=64
17 15.043783513 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=18/4608, ttl=64
18 16.044070034 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=19/4864, ttl=64
19 17.047471223 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=20/5120, ttl=64
20 18.052391611 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=21/5376, ttl=64
21 19.054236216 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=22/5632, ttl=64
22 20.060411703 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=23/5888, ttl=64
23 21.061115047 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=24/6144, ttl=64
24 22.065255024 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=25/6400, ttl=64
25 23.068913314 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=26/6656, ttl=64
26 23.214271551 PCSSystemtec_3f:bd:d2 → PCSSystemtec_1f:b7:23 ARP 60 192.168.56.102 is at 08:00:27:3f:bd:d2
27 24.072688340 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=27/6912, ttl=64
28 25.075344587 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=28/7168, ttl=64
29 26.076730589 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=29/7424, ttl=64
30 27.080371517 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=30/7680, ttl=64
31 28.083023888 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=31/7936, ttl=64
32 29.085996260 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=32/8192, ttl=64
33 30.093683907 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=33/8448, ttl=64
34 31.092761066 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=34/8704, ttl=64
35 32.095497601 192.168.56.102 → 192.168.56.20 ICMP 98 Echo (ping) request  id=0x0006, seq=35/8960, ttl=64
```

The script then converts this raw PCAP into a structured CSV file, extracting specific routing details and destination ports to dynamically identify the protocol involved.

> *The green Kali terminal output showing the structured* `traffic.csv` *file)*

```
┌──(kali㉿kali)-[~/Downloads/junior AI soc analyst]
└─$ cat traffic.csv
frame.time_epoch,ip.src,ip.dst,ip.proto,tcp.dstport
"1774353963.527005288","192.168.56.102","192.168.56.20","1",
"1774353964.530126341","192.168.56.102","192.168.56.20","1",
"1774353965.532550483","192.168.56.102","192.168.56.20","1",
"1774353966.534932878","192.168.56.102","192.168.56.20","1",
"1774353966.675397427",,,,
"1774353967.537976268","192.168.56.102","192.168.56.20","1",
"1774353968.539965772","192.168.56.102","192.168.56.20","1",
"1774353969.542790188","192.168.56.102","192.168.56.20","1",
"1774353970.546676128","192.168.56.102","192.168.56.20","1",
"1774353971.548028372","192.168.56.102","192.168.56.20","1",
"1774353972.553001985","192.168.56.102","192.168.56.20","1",
"1774353973.554074113","192.168.56.102","192.168.56.20","1",
"1774353974.554882032","192.168.56.102","192.168.56.20","1",
"1774353975.559007533","192.168.56.102","192.168.56.20","1",
"1774353976.562095319","192.168.56.102","192.168.56.20","1",
"1774353977.566620272","192.168.56.102","192.168.56.20","1",
"1774353978.570788801","192.168.56.102","192.168.56.20","1",
"1774353979.571075322","192.168.56.102","192.168.56.20","1",
"1774353980.574476511","192.168.56.102","192.168.56.20","1",
"1774353981.579396899","192.168.56.102","192.168.56.20","1",
"1774353982.581241504","192.168.56.102","192.168.56.20","1",
"1774353983.587416991","192.168.56.102","192.168.56.20","1",
"1774353984.588120335","192.168.56.102","192.168.56.20","1",
"1774353985.592260312","192.168.56.102","192.168.56.20","1",
"1774353986.595918602","192.168.56.102","192.168.56.20","1",
"1774353986.741276839",,,,
"1774353987.599693628","192.168.56.102","192.168.56.20","1",
"1774353988.602349875","192.168.56.102","192.168.56.20","1",
"1774353989.603735877","192.168.56.102","192.168.56.20","1",
"1774353990.607376805","192.168.56.102","192.168.56.20","1",
"1774353991.610029176","192.168.56.102","192.168.56.20","1",
"1774353992.613001548","192.168.56.102","192.168.56.20","1",
"1774353993.620689195","192.168.56.102","192.168.56.20","1",
```

---

6. Phase 5: Alert Generation & AI Triage

The script analyzes the CSV data against predefined thresholds. Once the threshold is breached, it identifies the protocol and automatically formats the evidence into a JSON alert payload.

> *The green Kali terminal output showing the `cat alert.json` payload)*

```
┌──(kali㉿kali)-[~/Downloads/junior AI soc analyst]
└─$ cat alert.json
{
  "alert_id": "SOC-AD5B4057",
  "alert_type": "Network Reconnaissance / Scanning",
  "indicator_type": "ip",
  "indicator_value": "192.168.56.102",
  "source_host": "Ubuntu-Threat-Actor",
  "destination_host": "Kali-SOC-Target",
  "destination_ip": "192.168.56.20",
  "protocol": "ICMP",
  "evidence": {
    "packet_count": 48,
    "time_window_seconds": 100,
    "data_source": "traffic.pcap"
  },
  "analyst_question": "Is this ICMP activity expected or malicious?"
}
```

Finally, the Python script transmits this JSON payload to the Airia API. The AI processes the live event and returns the final Tier-1 analyst report directly to the terminal, completing the automation loop.

> *The green Kali terminal output showing the HTTP 200 status and the full AI JSON response)*

```
┌──(kali㉿kali)-[~/Downloads/junior AI soc analyst]
└─$ python3 soc_capture.py
[+] Capturing on eth0 for 100s
Capturing on 'eth0'
77
[+] Capture saved to traffic.pcap
[+] CSV created at traffic.csv

[+] Traffic volume per source IP and Protocol:

  192.168.56.102 -> ICMP: 48 packets

[!] Suspicious ICMP activity detected from: 192.168.56.102
[+] Alert JSON written to alert.json
[+] Sending alert to Airia Agent Execution API...
[+] Airia responded with status 200
[+] Airia Response JSON:
{
 "$type": "string",
 "result": "{\n \"alert_id\": \"SOC-AD5B4057\",\n \"threat_classification\": \"Network Reconnaissance / Scanning\",\n \"score_
calculation_scratchpad\": \"Base 0 + 10 (packet_count > 30) = 10\",\n \"risk_score\": 10,\n \"risk_level\": \"Low\",\n \"confid
ence_level\": \"Medium\",\n \"is_likely_false_positive\": false,\n \"mitre_mapping\": {\n   \"tactic\": \"Discovery\",\n   \"t
echnique_id\": \"T1046\",\n   \"technique_name\": \"Network Service Scanning\"\n },\n \"analysis_reasoning\": \"The alert indic
ates ICMP-based network reconnaissance from an internal host (192.168.56.102, source_host: Ubuntu-Threat-Actor) to another interna
l asset (192.168.56.20, destination_ip) within a 100-second window, with 48 packets observed. This aligns with Network Reconnaissa
nce / Scanning, but the activity is moderate in volume and uses ICMP (not a high-rate flood). The internal naming of the source ho
st and private IPs suggest this could be legitimate internal administrative activity or a misconfiguration; however, without confi
rmation of an approved task, it remains a cautious observation. There is no evidence of credential use, data transfer, or access a
ttempts that would raise higher-risk concerns.\",\n \"recommended_actions\": [\n   \"Monitor for repeated ICMP reconnaissance fr
om 192.168.56.102 to 192.168.56.20 and correlate with any approved admin tasks or change-window activities.\",\n   \"Verify with
asset owners or change management whether ICMP discovery activities are authorized for the source host; if not, consider adjusting
ICMP visibility or rate-limiting on the source segment.\"\n ],\n \"escalation_required\": false,\n \"executive_summary\": \"An
internal host performed modest ICMP ping activity to another internal asset. This could be legitimate discovery or a misconfigura
tion; immediate action is not required, but ongoing monitoring and verification of authorization is advised.\"\n }",
 "report": null,
 "isBackupPipeline": false,
 "executionId": "08693a40-b052-4f91-bb7b-3b9f2701e45e",
 "userInputId": "b4ed8764-9b5c-49eb-bfc6-9a917d09709f",
 "files": [],
 "images": []
}

[+] Workflow complete.
```

The proofs: